实验1： PyTorch 的基本操作实验报告

姓名： 李韶庸
班级： 2024215122
学号： 2024213672

一、实验目的

1. 熟悉 PyTorch 的 Tensor 基本操作及数据类型转换。
2. 理解 Autograd 自动求导机制，例如 `requires_grad`、`backward()`、`zero_grad()` 等。
3. 掌握使用 `TensorDataset` 和 `DataLoader` 进行批量数据组织与加载。
4. 能够编写规范的深度学习训练流程，包括 `train / eval` 模式切换、Loss 记录、指标计算以及模型保存与加载。

二、实验数据与网络结构

2.1 实验数据

本实验采用 `sklearn` 自带的小型图像数据集 **load_digits()**。

数据特点：

- 输入特征：64维
- 标签类别：10类（0–9 的手写数字）

2.2 网络结构

构建了一个简单的 **两层多层感知机（MLP）**。

网络结构：

输入层 (64) → 隐藏层 → 输出层 (10)

具体配置：

- 隐藏层激活函数：ReLU
- 损失函数：CrossEntropyLoss
- 优化器：Adam

三、实验步骤

3.1 数据预处理

1. 读取数据后按比例划分：

- 训练集：约70%
- 验证集：约15%
- 测试集：约15%

2. 使用 `StandardScaler`：

- 在训练集上拟合特征分布
- 对训练集、验证集和测试集进行标准化处理

3.2 数据封装与加载

数据处理步骤：

1. 将特征矩阵转换为 `torch.float32` 类型 Tensor
2. 将标签转换为 `torch.long` 类型 Tensor（满足 CrossEntropyLoss 的要求）

然后使用：

- `TensorDataset` 组合特征与标签
- `DataLoader` 进行数据加载

参数设置：

训练集

- batch_size = 64
- shuffle = True

验证集 / 测试集

- batch_size = 256
- shuffle = False

3.3 模型训练与验证

训练共进行 **20 个 Epoch**。

每次训练步骤如下：

1. `optimizer.zero_grad()` 清空梯度
2. 前向传播计算输出
3. 计算 Loss
4. `loss.backward()` 反向传播
5. `optimizer.step()` 更新模型参数

每个 Epoch 结束后：

- 使用 `model.eval()` 切换至评估模式
- 在 `torch.no_grad()` 环境下计算验证集准确率

3.4 测试与模型保存加载

训练结束后执行以下步骤：

1. 在测试集上评估最终 **Test Accuracy**
2. 保存模型参数

```
torch.save(model.state_dict(), "mlp_model.pth")
```

3. 重新创建模型并加载参数

```
model = SimpleMLP()
model.load_state_dict(torch.load("mlp_model.pth"))
```

4. 再次在测试集上评估准确率，并与训练后的模型结果进行对比。

四、实验结果与分析

4.1 训练损失曲线（Training Loss Curve）

在训练过程中记录每个 Epoch 的平均训练损失，并绘制训练损失曲线。

结果分析：

从曲线可以观察到：

- 在训练初期，Loss 下降非常迅速
  从约 **2.0** 迅速下降到 **0.5 以下**
- 随着训练进行，Loss 下降速度逐渐减缓
- 最终稳定在 **约 0.05 左右**

这表明：

- 模型能够有效学习数据特征
- 优化器参数设置合理
- 训练过程稳定，没有出现梯度爆炸或梯度消失现象

4.2 模型一致性验证

在加载保存的模型权重后：

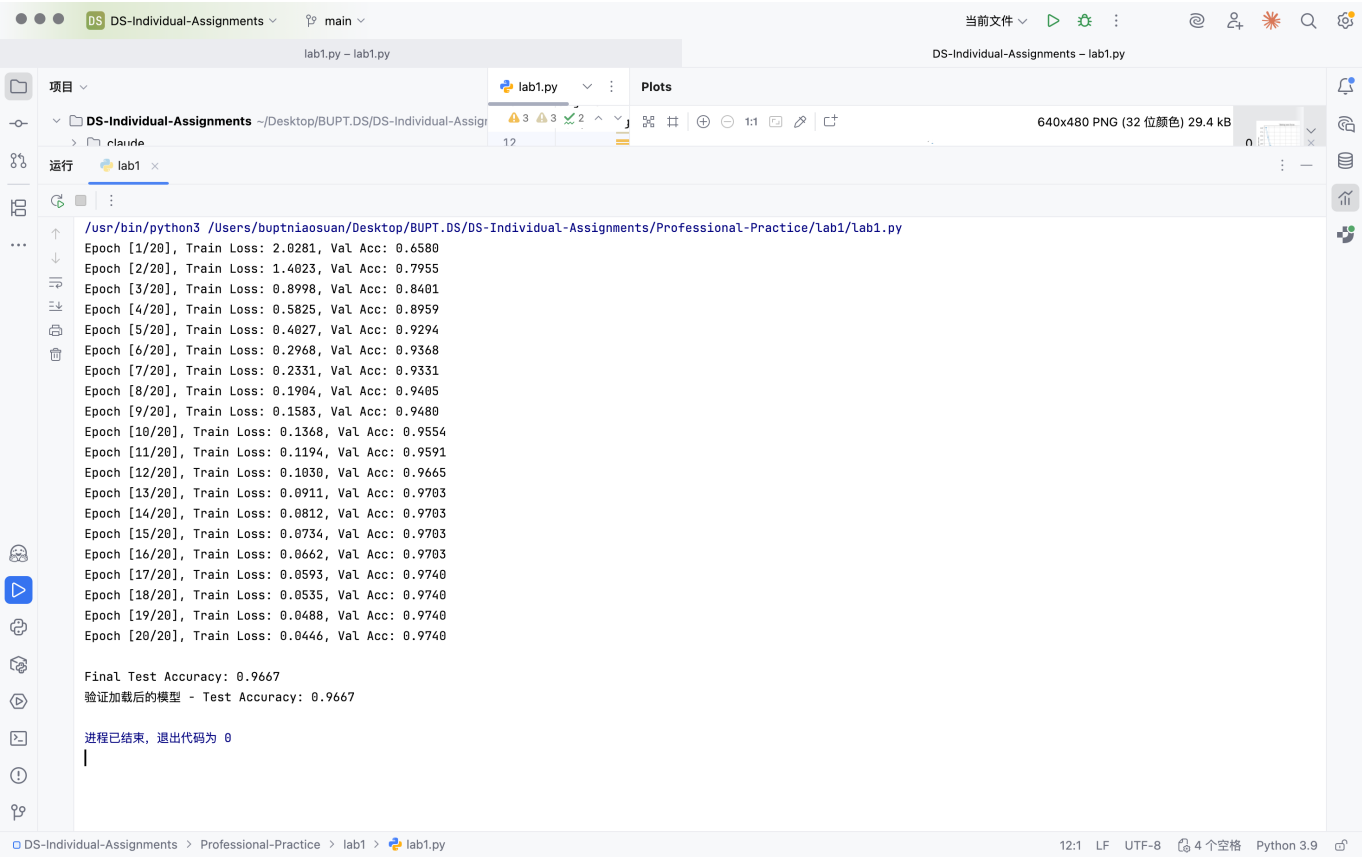
- 得到的 **Test Accuracy**
- 与训练完成后的模型结果 **完全一致**

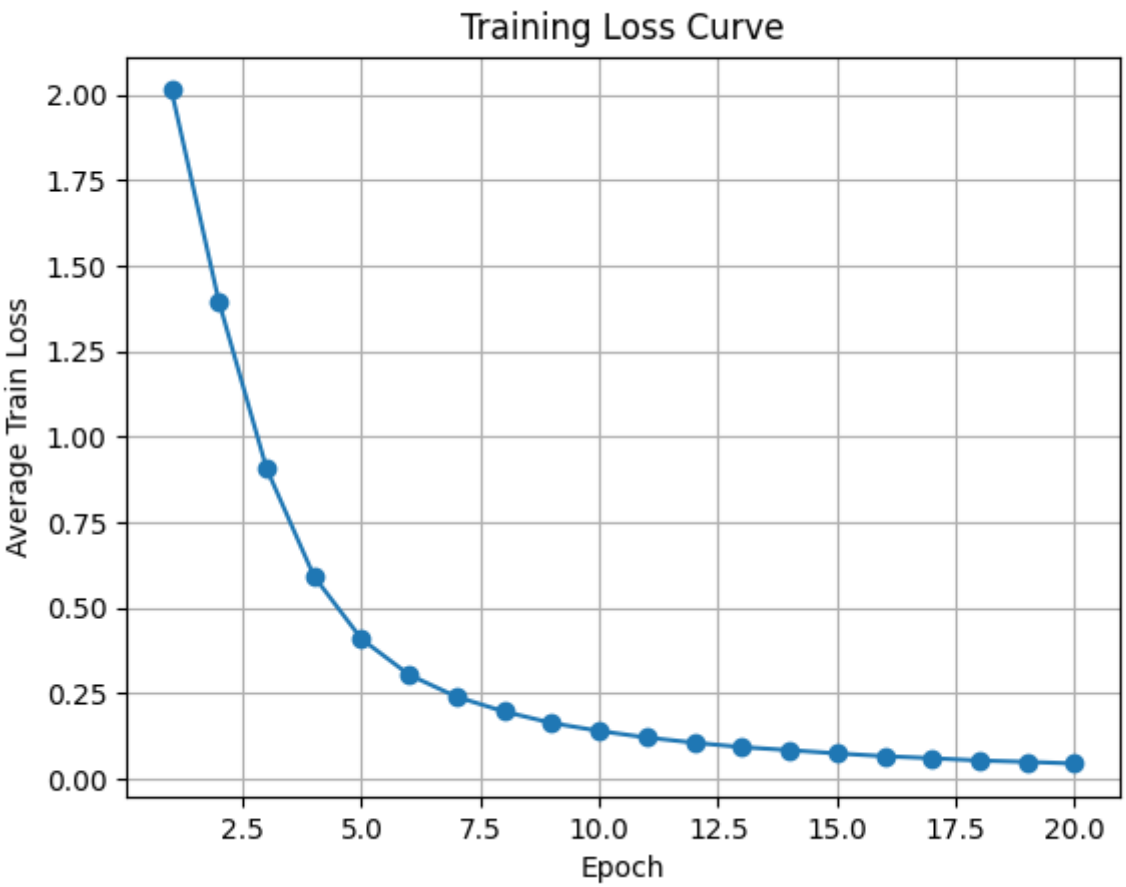
说明：

1. PyTorch 的 `state_dict` 可以完整保存模型参数
2. 加载模型后能够恢复原始模型状态
3. 使用 `model.eval()` 能保证推理阶段行为一致

五、实验结果图像

Training Loss Curve




Training Loss Curve
Average Train Loss
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epoch